

RELATÓRIO TÉCNICO — DESAFIO TEÓRICO-PRÁTICO (DTP)

Disciplina: Projeto de Software

Tema: Opção B — Streaming de Música (Microserviço de Catálogo + Microserviço de Reprodução)

Repositório: github.com/RianRBPS/projeto-software-dtp

Integrantes: Rian R. Santana

Junho de 2026

1. CONTEXTO E RESTRIÇÕES DO SISTEMA

O trabalho foi desenvolvido com base no **Tema B — Streaming de Música**, composto por dois microserviços isolados: o **ms-catalogo**, responsável pelo acervo de faixas, álbuns e artistas, e o **ms-reproducao**, responsável pela gestão do player, da fila de reprodução e do histórico do usuário. Toda a gerência de estado é realizada estritamente em memória, sem qualquer banco de dados relacional ou NoSQL.

1.1. Restrições Tecnológicas

- **Linguagem:** Java, Python ou JavaScript (Node.js) — a definir na Fase IV.
- **Persistência:** proibido o uso de banco de dados. Todo estado é mantido em coleções locais em memória (listas, mapas, etc.).
- **Infraestrutura:** cada microserviço possui seu próprio `Dockerfile`; a orquestração é feita via `docker-compose.yml`, que configura a rede interna e expõe as portas HTTP.

1.2. Regras de Negócio e Mapeamento de Padrões

RN	Descrição	Padrão GoF (Fase III)
RN01	A fila de reprodução agrega faixas de fontes heterogêneas (lista de álbum, grafo de recomendações, cache de histórico). O mecanismo de reprodução deve ser agnóstico quanto à estrutura de dados de origem.	Iterator
RN02	O algoritmo de seleção da próxima faixa deve ser intercambiável em tempo de execução: Modo Sequencial, Modo Shuffle (sem repetição no ciclo) e Modo Smart Mix (menor diferença de BPM).	Strategy

RN	Descrição	Padrão GoF (Fase III)
RN03	Alterações no estado do player (faixa atual, posição em ms, play/pause) devem notificar assincronamente o Módulo de Telemetria de Dispositivos e o Módulo de Histórico em Tempo Real.	Observer

2. FASE I — MODELAGEM CONCEITUAL (ANÁLISE DE DOMÍNIO)

A Fase I aplica a abordagem de Larman para construção do **Modelo de Domínio**: apenas classes conceituais, atributos em linguagem natural e associações com multiplicidade e verbos de ligação. É terminantemente proibido incluir assinaturas de métodos, modificadores de acesso (+/-) ou tipos de dados de linguagem de programação.

2.1. Classes Conceituais Identificadas

Classe	Atributos	Papel no Domínio
Usuario	nome, email	Titular da conta; controla o player e acumula histórico
Player	status	Componente central de reprodução; gerencia fila e estado
EstadoPlayer	posicaoMs, statusReproducao	Snapshot do instante de reprodução (faixa, posição, play/pause)
FilaReproducao	ordem	Agrega faixas de origens distintas; alimenta o player
Faixa	titulo, duracao, bpm	Unidade de conteúdo; possui metadados para o Smart Mix
Album	titulo, anoLancamento	Fonte linear de faixas; estrutura de lista
Artista	nome, genero	Autor de álbuns
GrafoRecomendacao	pontuacaoSimilaridade	Fonte de faixas via nós conectados por similaridade
HistoricoReproducao	dataHora	Registro das faixas ouvidas; base para retrospectiva

Classe	Atributos	Papel no Domínio
DispositivoUsuario	tipo, identificador	Celular, TV — receptores de telemetria do player

2.2. Associações Principais

Origem	Verbo	Destino	Multiplicidade
Usuario	controla	Player	1 → 1
Player	mantém	EstadoPlayer	1 → 1
Player	gerencia	FilaReproducao	1 → 1
FilaReproducao	agrega	Faixa	1 → 0..*
Album	contém	Faixa	1 → 1..*
Artista	lança	Album	1 → 1..*
GrafoRecomendacao	sugere	Faixa	1 → 0..*
Usuario	acumula	HistoricoReproducao	1 → 0..*
HistoricoReproducao	registra	Faixa	1 → 0..*
Usuario	usa	DispositivoUsuario	1 → 1..*

2.3. Diagrama de Classes de Análise

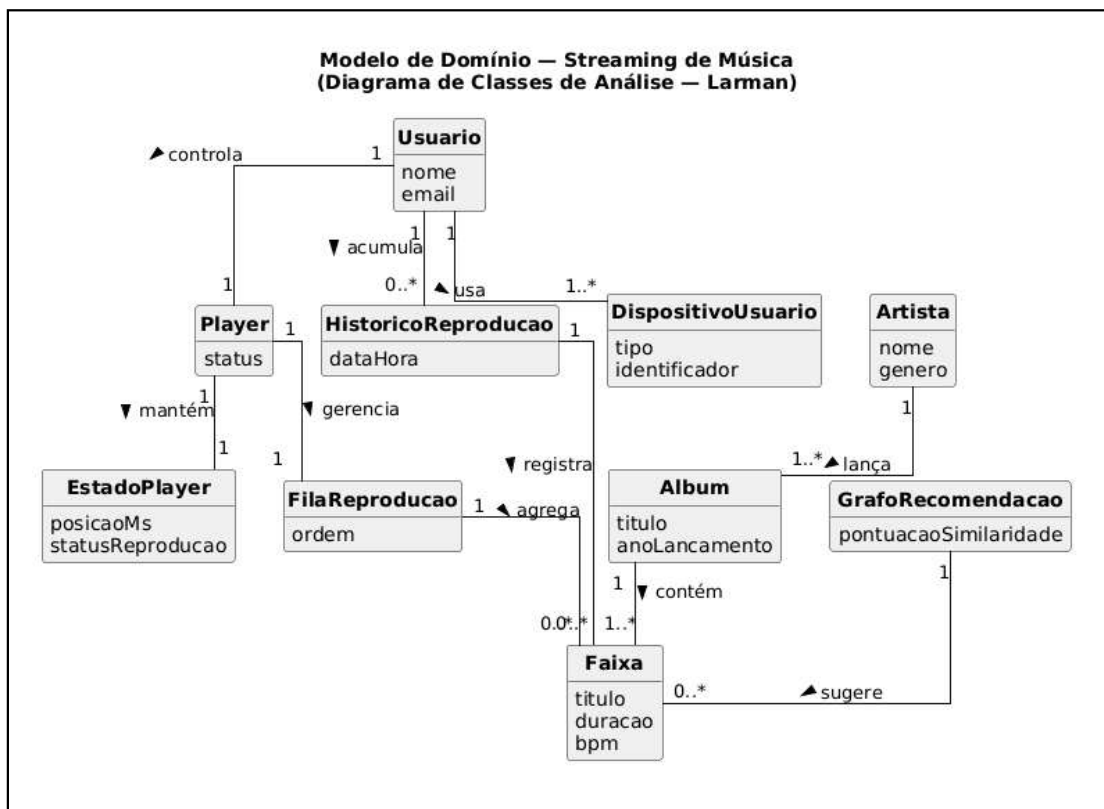


Figura 1 — Modelo de Domínio seguindo o padrão Larman (sem métodos, modificadores de acesso ou tipos de linguagem)

3. FASE II — MODELAGEM DE ARQUITETURA (MICROSERVIÇOS)

O sistema é decomposto em dois microserviços isolados, cada um com sua própria memória interna e sem compartilhamento de estado. A orquestração é feita via `docker-compose.yml`, que configura a rede interna `ntp-net` e expõe as portas HTTP ao cliente externo.

3.1. Decomposição em Microserviços

Container	Porta	Responsabilidades	Estado em memória
ms-catalogo	8080	CRUD de Faixas, Albums, Artistas e GrafoRecomendacao	List<Faixa>, List<Album>, List<Artista>, GrafoRecomendacao
ms-reproducao	8081	Gestão do Player, FilaReproducao, EstadoPlayer, HistoricoReproducao e Dispositivos	Player, FilaReproducao, EstadoPlayer, List<HistoricoReproducao>

3.2. Protocolo de Comunicação

Origem	Destino	Tipo	Exemplo de Rota	Propósito
ms-reproducao	ms-catalogo	HTTP GET síncrono	GET /faixas/{id}	Buscar dados de faixas para popular a fila
ms-reproducao	ModuloTelemetria	HTTP POST assíncrono	POST /telemetria/estado	Notificar celular e TV do usuário (Observer RN03)
ms-reproducao	ModuloHistorico	HTTP POST assíncrono	POST /historico/registrar	Computar dados para a retrospectiva (Observer RN03)

Isolamento de memória: nenhum objeto é compartilhado entre containers. O `ms-reproducao` mantém apenas o ID da faixa em sua fila; os dados completos (título, BPM etc.) são buscados sob demanda no `ms-catalogo` via HTTP GET síncrono.

3.3. Diagrama de Componentes

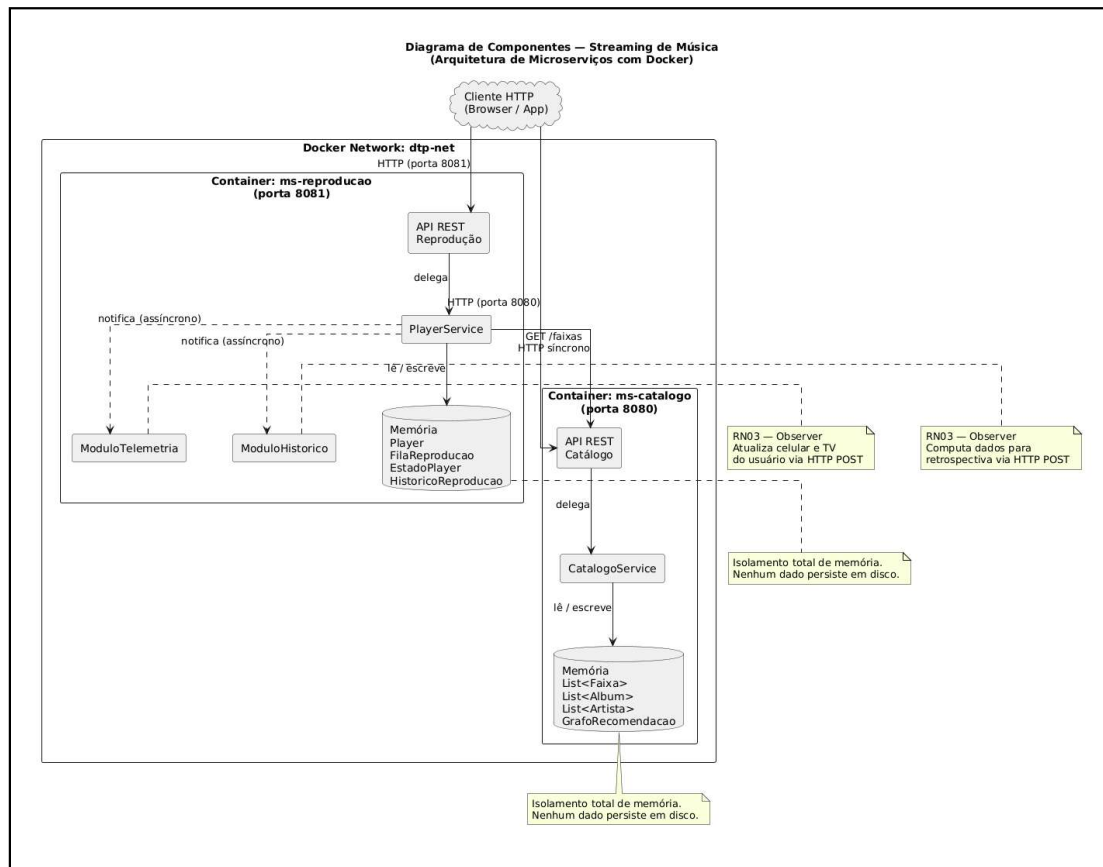


Figura 2 — Diagrama de Componentes: isolamento de memória por container e comunicação REST/HTTP na rede Docker

4. FASE III — DESIGN DETALHADO (PADRÕES DE PROJETO GOF)

O Diagrama de Classes de Projeto (DCD) é a evolução direta do Modelo de Domínio da Fase I. Nesta fase, todos os atributos recebem tipos de dados, os métodos ganham assinaturas completas com parâmetros, a visibilidade é explicitada (+ / -) e os três padrões GoF são aplicados conforme o mapeamento mandatório do DTP. O DCD é apresentado em quatro diagramas complementares para garantir legibilidade.

4.1. Padrão Iterator — RN01: Fila de Reprodução Agnóstica

A RN01 exige que o componente que consome a fila de reprodução seja completamente agnóstico quanto à estrutura de dados interna de cada fonte — lista linear de álbum, grafo de recomendações ou cache de histórico. O padrão **Iterator** resolve isso encapsulando a travessia atrás de uma interface uniforme.

- **Interface FaixaIterator:** define `hasNext()`, `next()` e `reset()` — única interface que o cliente (`Player`) conhece e utiliza.
- **AlbumIterator:** itera sobre `List<Faixa>` de um álbum por acesso indexado.
- **GrafoIterator:** percorre os nós do grafo de recomendações controlando um conjunto de visitados.
- **CacheIterator:** itera sobre a lista linear do cache de histórico de busca.
- **FilaReproducao (contexto/agregado):** aceita fontes distintas e delega a travessia ao iterator configurado. O cliente jamais acessa `List<Faixa>`, `List<NoGrafo>` ou qualquer estrutura interna das fontes.

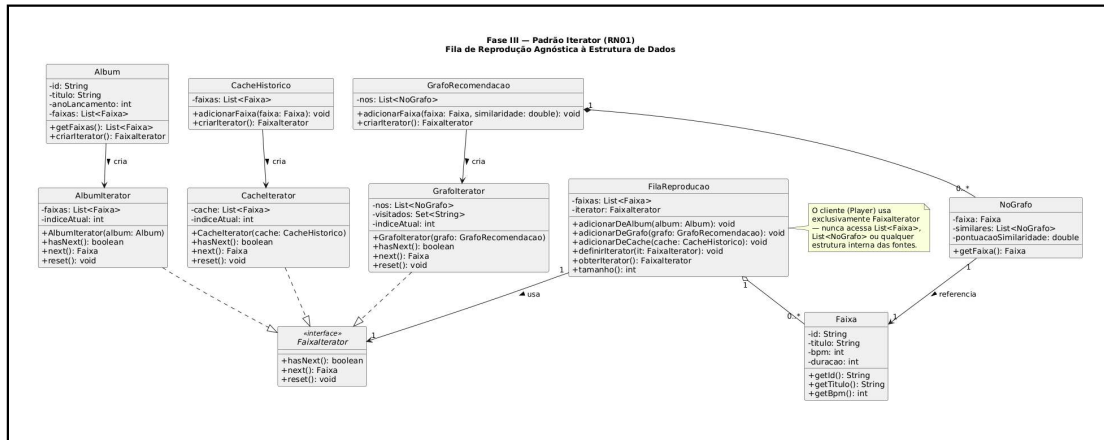


Figura 3 — DCD do padrão Iterator: FaixaIterator e os três iteradores concretos

4.2. Padrão Strategy — RN02: Algoritmo de Seleção Intercambiável

A RN02 exige que o comportamento de seleção da próxima faixa mude dinamicamente, sem reiniciar o container. O padrão **Strategy** encapsula cada algoritmo em uma classe concreta intercambiável em tempo de execução via `Player.definirEstrategia()`.

- **Interface EstrategiaSelecao:** contrato único — `selecionarProxima(fila, historico): Faixa`.
- **ModoSequencial:** segue estritamente a ordem de inserção na fila, esvaziando-a progressivamente.
- **ModoShuffle:** seleciona aleatoriamente sem repetir faixas no ciclo atual; mantém conjunto de já tocadas.
- **ModoSmartMix:** varre a fila e seleciona a faixa com menor diferença de BPM em relação à faixa atual, priorizando continuidade do ritmo.
- **Player (contexto):** delega a seleção à estratégia configurada; `definirEstrategia()` permite a troca em runtime — evidenciando o princípio Open/Closed.

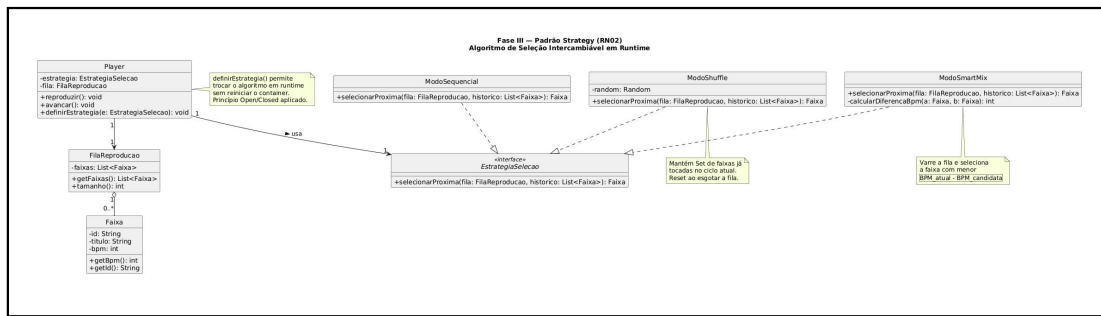


Figura 4 — DCD do padrão Strategy: EstrategiaSelecao e os três modos concretos

4.3. Padrão Observer — RN03: Sincronização de Estado

A RN03 exige que mudanças no `EstadoPlayer` sincronizem automaticamente e de forma assíncrona dois módulos independentes. O padrão **Observer** desacopla o `Player` (Subject) dos módulos receptores (Observers).

- **Interface ObservadorEstado:** contrato único — `atualizar(estado: EstadoPlayer): void`.
- **ModuloTelemetria:** recebe o estado e envia HTTP POST assíncrono para cada `DispositivoUsuario` (celular, TV).
- **ModuloHistorico:** recebe o estado e persiste a faixa em `HistoricoReproducao` em memória.
- **Player (Subject):** mantém lista de observers; invoca `notificarObservadores()` internamente em toda mudança de estado.

Justificativa: Push vs. Pull

Adotou-se o modelo **Push**: o `Player` envia o `EstadoPlayer` completo a cada observer no momento da notificação. A escolha se justifica por três razões:

1. **Redução de round-trips de rede:** no modelo Pull, cada observer precisaria fazer uma requisição adicional ao `Player` para buscar o estado — dobrando o tráfego entre microserviços.
2. **Interface autocontida:** o contrato `atualizar(estado)` carrega tudo que o observer precisa; ele não mantém referência ao Subject nem conhece sua API de leitura, reduzindo o acoplamento estrutural.
3. **Trade-off aceito:** o acoplamento de dados cresce — qualquer campo adicionado a `EstadoPlayer` é propagado a todos os observers, mesmo que não o utilizem. Para o escopo deste sistema, onde `EstadoPlayer` possui apenas três campos (`faixaAtual`, `posicaoMs`, `statusReproducao`), esse custo é desprezível e o benefício de desempenho de rede supera o aumento de acoplamento.

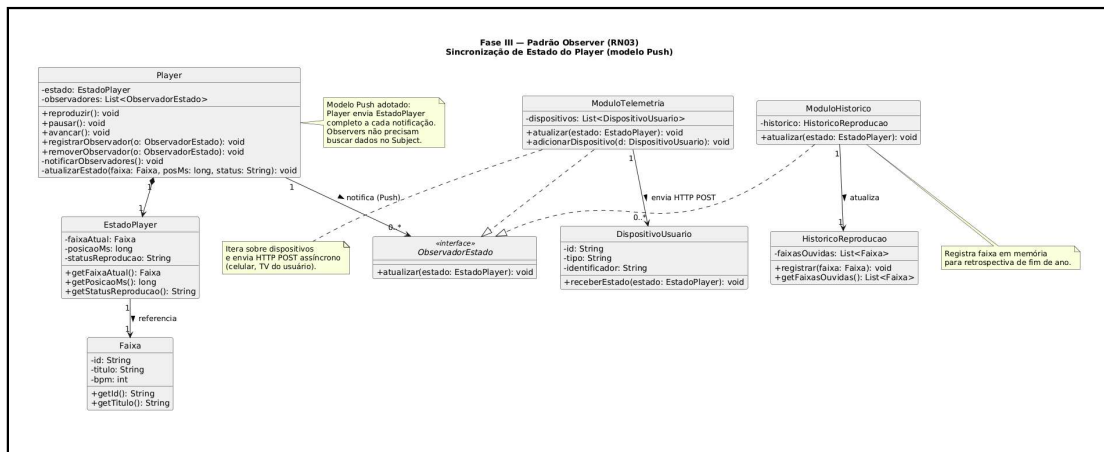


Figura 5 — DCD do padrão Observer: Player como Subject notificando ModuloTelemetria e ModuloHistorico (modelo Push)

4.4. Visão Geral do DCD

O diagrama a seguir apresenta a visão consolidada do DCD, mostrando o `Player` como contexto central que integra os três padrões aplicados, sem detalhar as implementações concretas de cada um.

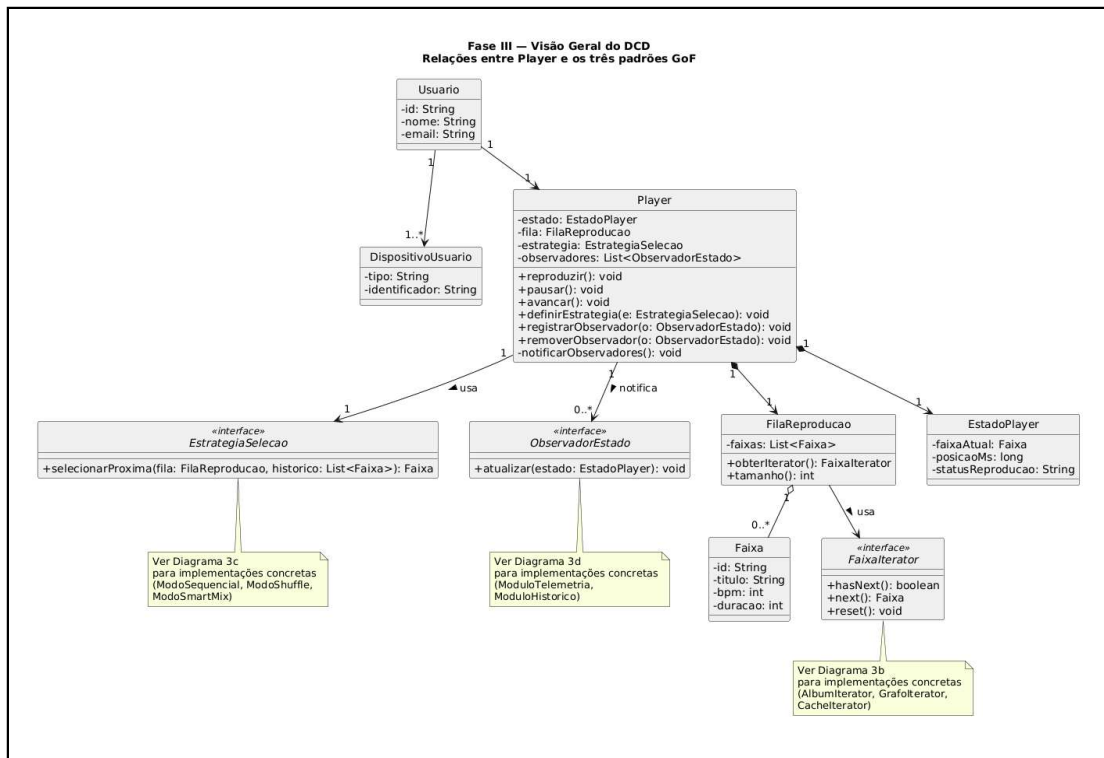


Figura 6 — Visão geral do DCD: Player integrando FaixaIterator (Iterator), EstrategiaSelecao (Strategy) e ObservadorEstado (Observer)

5. ITENS OBRIGATÓRIOS DA APRESENTAÇÃO

5.1. Demonstração Dinâmica do Strategy

A equipe deverá executar um cenário de teste ou rota de API criando uma nova estratégia de algoritmo não prevista na modelagem inicial (ex: **Modo Cronológico**

— ordena faixas por ano de lançamento do álbum) e realizar a troca dessa estratégia via chamada HTTP em tempo de execução, modificando o comportamento do sistema sem reiniciar o container, evidenciando o princípio Open/Closed.

5.2. Defesa Científica do Observer (Push vs. Pull)

Conforme justificado na Seção 4.3, a equipe adotou o modelo Push. A defesa deverá discutir formalmente:

- O impacto da escolha no acoplamento das classes;
- O impacto no tráfego de rede entre os microserviços;
- A evolução a longo prazo do sistema sob cada modelo.

5.3. Encapsulamento do Iterator

A equipe deverá mostrar no código-fonte que o cliente que lê os dados da RN01 faz uso exclusivo da interface `FaixaIterator`, estando completamente blindado e incapaz de acessar ou conhecer a coleção interna (`List<Faixa>`, `List<NoGrafo>` etc.) que guarda os dados em memória.